

ASSIGNMENT – 9.3

2303A51355

Batch-10

Task-1

Prompt: act as a auto generating function documentation in a shared codebase.

#in a python script with multiple functions without any doc strings. now uh generate google-style function docstrings for each function in the script.

The docstrings should include a brief description of the function's purpose, its parameters, and its return value.

take user input for the functions and print the results.

code :

```
from typing import Required
```

```
def add(a, b):
```

```
    return a + b
```

```
def subtract(a, b):
```

```
    return a - b
```

```
def multiply(a, b):
```

```
    return a * b
```

```
def divide(a, b):
```

```
    if b == 0:
```

```
        raise ValueError("Cannot divide by zero.")
```

```
    return a / b
```

```
# User input
```

```
try:
```

```
    num1 = float(input("Enter the first number:
```

```
"))
```

```
    num2 = float(input("Enter the second
```

```
number: "))
```

```

print(f"Addition: {add(num1, num2)}")

print(f"Subtraction: {subtract(num1,
num2)}")

print(f"Multiplication: {multiply(num1,
num2)}")

print(f"Division: {divide(num1, num2)}")

except ValueError as e:

print(e)

```

The screenshot shows the Visual Studio Code editor with a file named `lab-9.3.py` open. The code defines four functions: `add`, `subtract`, `multiply`, and `divide`. The `divide` function includes a try-except block to handle a `ValueError` when dividing by zero. The program prompts the user for two numbers and prints the results of all four operations. The terminal output shows the program running successfully with the following results:

```

PS C:\Users\sai\r\OneDrive\Desktop\AI assistant> & 'c:\Users\sai\r\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\sai\r\OneDrive\Desktop\AI assistant\lab-9.3.py'
Enter the first number: 4
Enter the second number: 5
Addition: 9.0
Subtraction: -1.0
Multiplication: 20.0
Division: 0.8

```

Code Analysis:

- ☐ The script defines four arithmetic functions: `add`, `subtract`, `multiply`, and `divide`.
- ☐ Each function performs a basic mathematical operation on two numbers.
- ☐ The `divide()` function includes error handling to prevent division by zero.
- ☐ The program takes user input for two numbers and calls all functions to display results.
- ☐ Google-style docstrings help improve readability and maintainability in shared codebases.

Task-2

Prompt: **act as ai generated inline comments for a complex codebase.**

#in a python script with complex code without any comments.now uh generate inline comments for the code to explain the logic and flow of the program.

#use loops and conditional logic and take user input.

Code :

```
def factorial(n):  
    if n < 0:  
        return "Factorial is not defined for negative numbers."  
    elif n == 0 or n == 1:  
        return 1  
    else:  
        result = 1  
        # Loop from 2 to n to calculate the factorial  
        for i in range(2, n + 1):  
            result *= i  
        return result  
  
# User input  
try:  
    number = int(input("Enter a number to calculate its factorial: "))  
    print(f"The factorial of {number} is: {factorial(number)}")  
except ValueError:  
    print("Please enter a valid integer.")
```

The screenshot shows the Visual Studio Code interface. The Explorer panel on the left lists files in the 'AI ASSISTANT' workspace, including 'lab-9.3.py'. The main editor displays the code for 'lab-9.3.py', which defines a 'factorial' function and uses it to calculate the factorial of a user input. The code includes comments explaining the logic. The Terminal panel at the bottom shows the command prompt running the script, entering the number 5, and displaying the output 'The factorial of 5 is: 120'.

```
30 #in a python script with complex code without any comments.now un generate inline comments for the code to
31 #use loops and conditional logic and take user input
32 def factorial(n):
33     if n < 0:
34         return "Factorial is not defined for negative numbers."
35     elif n == 0 or n == 1:
36         return 1
37     else:
38         result = 1
39         # Loop from 2 to n to calculate the factorial
40         for i in range(2, n + 1):
41             result *= i
42         return result
43 # User input
44 try:
45     number = int(input("Enter a number to calculate its factorial: "))
46     print(f"The factorial of {number} is: {factorial(number)}")
47 except ValueError:
48     print("Please enter a valid integer.")
49 """
```

PS C:\Users\saipr\OneDrive\Desktop\AI assistant> & 'c:\Users\saipr\AppData\Local\Programs\Python\Python313\python.exe' '18.0-win32-x64\bundled\libs\debugpy\launcher' '62637' '--' 'c:\Users\saipr\OneDrive\Desktop\AI assistant\lab-9.3.py'
Enter a number to calculate its factorial: 5
The factorial of 5 is: 120

Code Analysis:

- ☐ The script calculates the factorial of a number using a function.
- ☐ It uses conditional statements to handle negative numbers and base cases (0 and 1).
- ☐ A loop is used to multiply numbers from 2 to n to compute the factorial.
- ☐ The program takes user input and validates it using try–except for invalid integers.
- ☐ Inline comments explain the logic and flow of the program step-by-step.

Task-3

Prompt: Write a professional multi-line module-level docstring for the given Python module.

#The docstring should clearly describe the purpose of the module, required dependencies, key classes and functions, and include a short example of how the module can be used.

#Ensure the tone is suitable for an internal repository or real-world project.

Code :

class Student:

```
def __init__(self, student_id, name, marks):  
  
    self.student_id = student_id  
  
    self.name = name  
  
    self.marks = marks
```

```
def calculate_average(self):
    if not self.marks:
        return 0
    return round(sum(self.marks) / len(self.marks), 2)

def add_student(student_list, student):
    student_list.append(student)

def find_student_by_id(student_list, student_id):
    for student in student_list:
        if student.student_id == student_id:
            return student
    return None

if __name__ == "__main__":
    print("===== Student Management Demo =====")
    students = []
    sid = int(input("Enter Student ID: "))
    name = input("Enter Student Name: ")
    marks_input = input("Enter marks separated by space: ")
    marks = list(map(int, marks_input.split()))

    student = Student(sid, name, marks)
    add_student(students, student)

    search_id = int(input("Enter Student ID to search: "))
    result = find_student_by_id(students, search_id)

    if result:
        print("Student Found:", result.name)
        print("Average Marks:", result.calculate_average())
```

```
print("Student not found")
```



- ### Task-4

#Preserve the original meaning of the comments, include description, parameters, return values, and an example where appropriate.

Code :

```
class ShoppingCart:
```

```

def __init__(self):
    self.cart = {}

def add_item(self, item_name, price):
    if item_name in self.cart:
        self.cart[item_name] += price
    else:
        self.cart[item_name] = price

def remove_item(self, item_name):
    if item_name in self.cart:
        del self.cart[item_name]
    else:
        print("Item not found in cart.")

def calculate_total(self):
    return sum(self.cart.values())

# User input
cart = ShoppingCart()

while True:
    action = input("Enter 'add' to add an item, 'remove' to
remove an item, 'total' to calculate total price, or 'exit' to quit:
")

    if action == 'add':
        item_name = input("Enter the name of the item: ")
        try:
            price = float(input("Enter the price of the item: "))
            cart.add_item(item_name, price)
            print(f"Added {item_name} to cart.")
        except ValueError:
            print("Invalid price. Please enter a number.")

```

```

elif action == 'remove':

    item_name = input("Enter the name of the item to
remove: ")

    cart.remove_item(item_name)

elif action == 'total':

    total_price = cart.calculate_total()

    print(f"The total price of items in the cart is:
${total_price:.2f}")

elif action == 'exit':

    print("Exiting the shopping cart program.")

    break

else:

    print("Invalid action. Please try again.")

```

Output:

The screenshot shows the Visual Studio Code interface. The Explorer pane on the left lists files in a project named 'AI ASSISTANT', including PDFs, logs, and various Python files. The file 'lab-9.3.py' is selected and open in the editor. The code in the editor defines two functions: `calculate_discount` and `find_max`, followed by a `__main__` block that prompts the user for input and prints the results. The TERMINAL pane at the bottom shows the command prompt output, which matches the code's logic: it prompts for an original price (40), a discount percentage (4), calculates the final price (38.4), prompts for numbers (1 2 3 4 5), and prints the maximum value (5.0).

```

91 #Preserve the original meaning of the comments, include description, parameters, return values, and an example where appropriate.
92 #Remove redundant inline comments after converting them into docstrings to keep the function bodies clean.
93 def calculate_discount(price, discount_percentage):
94     if price < 0:
95         return 0
96     discount = (price * discount_percentage) / 100
97     final_price = price - discount
98     return final_price
99 def find_max(numbers):
100     if not numbers:
101         return None
102     maximum = numbers[0]
103     for num in numbers:
104         if num > maximum:
105             maximum = num
106     return maximum
107 if __name__ == "__main__":
108     print("==== Discount Calculator & Maximum Finder ====")
109     # User input for discount calculation
110     price = float(input("Enter original price: "))

```

```

PS C:\Users\sai\r\OneDrive\Desktop\AI assistant> & 'c:\Users\sai\r\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\sai\r\.vscode\
18.0-win32-x64\bundled\libs\debugpy\launcher' '49677' '--' 'c:\Users\sai\r\OneDrive\Desktop\AI assistant\lab-9.3.py'
==== Discount Calculator & Maximum Finder ====
Enter original price: 40
Enter discount percentage: 4
Final Price after Discount: 38.4
Enter numbers separated by space: 1 2 3 4 5
Maximum Value: 5.0

```

Code Analysis :

- ☐ The script contains two functions: `calculate_discount()` and `find_max()`.
- ☐ `calculate_discount()` computes the final price after applying a discount.

- ❑ `find_max()` finds the maximum value from a list of numbers using a loop.
- ❑ The program takes user input for price, discount percentage, and numbers list.
- ❑ Converting inline comments to Google-style docstrings improves documentation structure and keeps function bodies clean.

Task-5

Prompt: Design a Python utility script that reads a .py file and automatically detects functions and classes.

#Insert placeholder Google-style docstrings for each detected function or class.

#The focus is on documentation scaffolding, not perfect documentation.

#Show how AI assistance can be used to generate or refine this tool.

```
import re
import os

def generate_docstrings(file_path):
    if not os.path.isfile(file_path):
        print("File not found.")
        return

    with open(file_path, 'r') as file:
        content = file.read()

    # Regex patterns to detect classes and functions
    class_pattern = re.compile(r'class\s+(\w+)')
    function_pattern = re.compile(r'def\s+(\w+)\s*\(')

    classes = class_pattern.findall(content)
    functions = function_pattern.findall(content)

    # Generate placeholder docstrings
    for cls in classes:
        print(f'Class: {cls}\n"""[Class description here]"""\n')

    for func in functions:
        print(f'Function: {func}\n"""[Function description here]\nArgs:\n [parameters]\nReturns:\n [return value]"""\n')
```

```

if __name__ == "__main__":

    file_path = input("Enter the path of the .py file to analyze: ")

    generate_docstrings(file_path)

```

Output :

```

lab-9.3.py > generate docstrings
123 #Design a Python utility script that reads a .py file and automatically detects functions and classes.
124 #Insert placeholder Google-style docstrings for each detected function or class.
125 #The focus is on documentation scaffolding, not perfect documentation.
126 #Show how AI assistance can be used to generate or refine this tool.
127 import re
128 import os
129 def generate_docstrings(file_path):
130     if not os.path.isfile(file_path):
131         print("File not found.")
132         return
133     with open(file_path, 'r') as file:
134         content = file.read()
135     # Regex patterns to detect classes and functions
136     class_pattern = re.compile(r'class\s+(\w+)')
137     function_pattern = re.compile(r'def\s+(\w+)\s*\(')
138     classes = class_pattern.findall(content)
139     functions = function_pattern.findall(content)
140     # Generate placeholder docstrings
141     for cls in classes:
142         print(f'Class: {cls}\n""[Class description here]""\n')
143
144 PS C:\Users\sai\r\OneDrive\Desktop\AI assistant> & 'c:\Users\sai\r\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\sai\r\.vscode\extensions\ms-python.python\python\cli.py' 'lab-9.3.py'
Enter the path of the .py file to analyze: lab-9.3.py
Class: Student
""[Class description here]""

Function: add
""[Function description here]
Args:
    [parameters]
Returns:
    [return value]
""

Function: subtract
""[Function description here]

```

Code Analysis :

- ☐ The script reads a Python (.py) file provided by the user.
- ☐ It uses regular expressions (regex) to detect classes and functions in the file.
- ☐ The program automatically generates placeholder Google-style docstrings.
- ☐ It helps developers quickly create documentation scaffolding for large codebases.
- ☐ AI assistance can further refine the generated docstrings and improve documentation quality.